

● PROGRAMACIÓN AVANZADA · UTU · BACHILLERATO TECNOLÓGICO

Actividades de práctica – BD y Herencia

Tres actividades para consolidar la conexión a MySQL con JDBC, las 5 fases del acceso a datos y el patrón de herencia con dos tablas.

TEMA 3 – JDBC + MYSQL

TEMA 4 – HERENCIA + JOIN

PROYECTO BASE: MINI-CRUD-SQL

DIFICULTAD CRECIENTE

01

CONSOLIDACIÓN · TEMA 3 – JDBC

NIVEL BÁSICO

Agregar columna **fecha_nacimiento** a la tabla personas

Ampliar la entidad Persona para incluir el campo **fecha_nacimiento** (tipo DATE en MySQL). El cambio debe propagarse por todas las capas: la tabla en MySQL, el VO, los 5 métodos del DAO y las tres vistas. Esta actividad aplica las 5 fases del acceso a datos en todos los métodos CRUD y ejercita el manejo del tipo `java.sql.Date` en JDBC.

PROYECTO BASE
mini-crud-sqlTIPO SQL NUEVO
ALTER TABLE · DATE

// FASES DEL DAO QUE SE MODIFICAN

MÉTODO	SQL A ACTUALIZAR	PARÁMETRO NUEVO
<code>listarPersonas()</code>	SELECT codigo, nombre, fecha_nacimiento FROM personas	<code>rs.getDate("fecha_nacimiento")</code>
<code>agregarPersona()</code>	INSERT INTO personas(codigo, nombre, fecha_nacimiento)	<code>ps.setDate(3, ...)</code>
<code>actualizarPersona()</code>	UPDATE personas SET nombre=?, fecha_nacimiento =?	<code>ps.setDate(2, ...)</code>
<code>eliminarPersona()</code>	Sin cambios — DELETE no usa columnas	—
<code>obtenerPersonaPorCodigo()</code>	SELECT codigo, nombre, fecha_nacimiento WHERE codigo=?	<code>rs.getDate("fecha_nacimiento")</code>

// TAREAS

1

En MySQL, ejecutar el siguiente comando para agregar la columna a la tabla existente sin perder los datos que ya hay:

MySQL – agregar columna

```
-- Agregar columna fecha_nacimiento a la tabla existente
ALTER TABLE personas ADD COLUMN fecha_nacimiento DATE;
```

- 2 En **PersonaVO.java**: declarar `private java.sql.Date fechaNacimiento`, actualizar el constructor para recibirlo como tercer parámetro, y agregar getter y setter.
- 3 En **PersonaDAO.java**: actualizar los métodos `listarPersonas()`, `agregarPersona()`, `actualizarPersona()` y `obtenerPersonaPorCodigo()` según la tabla de arriba. Usar `rs.getDate()` para leer y `ps.setDate()` para escribir. El método `eliminarPersona()` no necesita cambios.
- 4 En **persona-form.jsp**: agregar un campo `<input type="date">` con `name="fechaNacimiento"`. Actualizar el Servlet para que lea el parámetro con `java.sql.Date.valueOf(request.getParameter("fechaNacimiento"))` y lo pase al VO.
- 5 En **persona-lista.jsp**: agregar la columna "Fecha de nacimiento" a la tabla y mostrar `${p.fechaNacimiento}` en cada fila.
- 6 En **persona-editar.jsp**: agregar el campo con `value="${persona.fechaNacimiento}"` para que aparezca precargado al editar.

// ARCHIVOS A MODIFICAR

Archivos del proyecto

Base de datos:

personas tabla → ALTER TABLE + columna fecha_nacimiento

Capa Modelo:

src/main/java/modelo/vo/PersonaVO.java

src/main/java/modelo/dao/PersonaDAO.java ← 4 de 5 métodos

Capa Controlador:

src/main/java/controlador/PersonaControladorServlet.java

Capa Vista:

src/main/webapp/WEB-INF/vista/persona-form.jsp

src/main/webapp/WEB-INF/vista/persona-lista.jsp

src/main/webapp/WEB-INF/vista/persona-editar.jsp

△ ORDEN DE LOS PARÁMETROS EN PREPAREDSTATEMENT

Los `?` se numeran según el orden en que aparecen en el SQL, empezando en 1. Si el SQL es `INSERT INTO personas(codigo, nombre, fecha_nacimiento) VALUES (?, ?, ?)`, entonces: `ps.setInt(1, codigo)`, `ps.setString(2, nombre)`, `ps.setDate(3, fechaNacimiento)`. Intercambiarlos produce errores silenciosos difíciles de detectar. Nota: en el Servlet usar `java.sql.Date.valueOf()` para convertir el String del formulario a `java.sql.Date`.

// CRITERIOS DE VERIFICACIÓN

- ☐ Al ingresar una persona con fecha de nacimiento, el dato se guarda y aparece en la lista con el formato correcto.
- ☐ Al editar, el campo fecha de nacimiento aparece precargado con el valor guardado en la BD.
- ☐ Modificar la fecha y guardar actualiza el dato en la BD (verificar con `SELECT * FROM personas` en MySQL).
- ☐ Eliminar una persona la borra correctamente y no genera error de SQL.

02

APLICACIÓN · TEMA 3 — CONSULTA PERSONALIZADA

NIVEL INTERMEDIO

Búsqueda de personas por nombre en la BD

Agregar un método nuevo al DAO que permita **filtrar personas por nombre** usando SQL LIKE. Integrar la búsqueda con una acción nueva en el Servlet y un formulario en la vista de lista, de modo que la tabla muestre solo los resultados que coincidan con lo buscado.

PROYECTO BASE | SQL NUEVO
mini-crud-sql | **LIKE con % (comodín)**

// CONCEPTO: SQL LIKE

```
SQL - búsqueda parcial con LIKE

-- El % actúa como comodín: cualquier cantidad de caracteres
SELECT * FROM personas WHERE nombre LIKE '%ana%';
-- devuelve: "Ana García", "Mariana López", "Fabiana Torres"

-- Con PreparedStatement (forma segura, sin concatenar el %):
String sql = "SELECT * FROM personas WHERE nombre LIKE ?";
ps.setString(1, "%" + termino + "%");
```

// TAREAS

- 1 En **PersonaDAO.java**: agregar un nuevo método `buscarPorNombre(String termino)` que retorne `List<PersonaVO>`. El SQL debe usar `LIKE ?` y el parámetro debe ser `"%" + termino + "%"` para buscar en cualquier parte del nombre.
- 2 En **PersonaControladorServlet.java**, en el `doGet`: agregar un nuevo `case "buscar"` que lea el parámetro `termino` del request, llame a `dao.buscarPorNombre(termino)`, guarde el resultado con `request.setAttribute("personas", ...)` y haga forward a `persona-lista.jsp`.
- 3 En **persona-lista.jsp**: agregar un pequeño formulario de búsqueda **encima de la tabla**, con un campo de texto (`name="termino"`), un botón "Buscar" y un enlace "Ver todos". El formulario debe enviar por GET a `action="buscar"`.
- 4 Si la búsqueda no devuelve resultados, mostrar un mensaje en la página: "No se encontraron personas con ese nombre." El mismo JSP se reutiliza para mostrar tanto la lista completa como los resultados filtrados.

// ARCHIVOS A MODIFICAR

 Archivos del proyecto

Capa Modelo:

`src/main/java/modelo/dao/PersonaDAO.java`→ agregar método `buscarPorNombre(String termino)`

Capa Controlador:

`src/main/java/controlador/PersonaControladorServlet.java`→ agregar case "buscar" en `doGet`

Capa Vista:

`src/main/webapp/WEB-INF/vista/persona-lista.jsp`

→ agregar formulario de búsqueda y mensaje de sin resultados

 POR QUÉ GET Y NO POST PARA LA BÚSQUEDA

Las búsquedas usan `method="get"` porque no modifican datos. Además, el término buscado queda en la URL (`?action=buscar&termino=ana`), lo que permite compartir o guardar el link de los resultados. El patrón PRG solo aplica a operaciones que escriben en la BD.

// CRITERIOS DE VERIFICACIÓN

- ☐ Buscar "an" muestra todas las personas cuyo nombre contiene esas letras.
- ☐ La búsqueda no distingue posición: funciona si "an" está al inicio, en el medio o al final del nombre.
- ☐ Si no hay coincidencias, se muestra el mensaje "No se encontraron personas" en lugar de una tabla vacía sin explicación.
- ☐ El enlace "Ver todos" vuelve al listado completo sin filtro.

03

INTEGRADOR · TEMA 4 – HERENCIA + TRANSACCIONES

NIVEL AVANZADO

CRUD completo para DocenteVO extends PersonaVO

Aplicar el mismo patrón de herencia del Tema 4 para crear un CRUD completo de **Docente**. La entidad Docente extiende PersonaVO (hereda *codigo* y *nombre*) y agrega **especialidad** y **cargo**. Los datos se distribuyen en dos tablas (*personas* y *docentes*) vinculadas por FK con ON DELETE CASCADE, igual que el modelo Alumno.

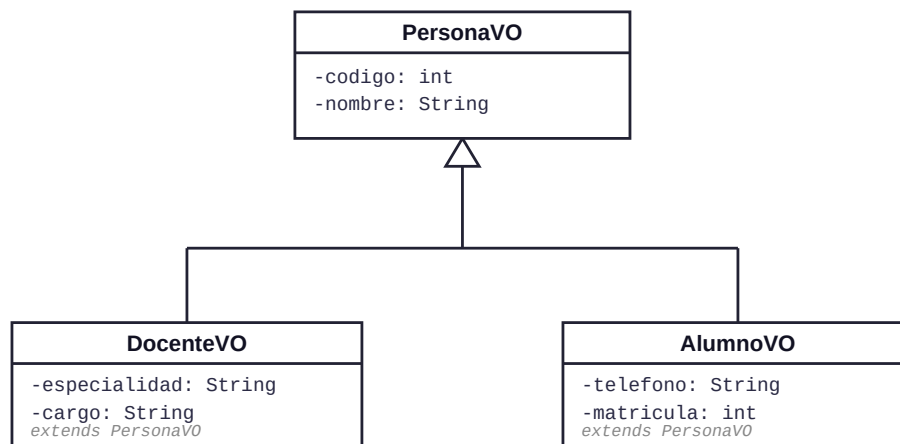
PROYECTO BASE

mini-crud-alumno-sql

PATRÓN APLICADO

2 tablas · JOIN · transacción

// MODELO DE DATOS



// TABLAS EN MYSQL

MySQL – crear tablas

```

-- La tabla personas ya existe; solo crear la tabla hija:
CREATE TABLE docentes (
  codigo      INT PRIMARY KEY,
  especialidad VARCHAR(100) NOT NULL,
  cargo       VARCHAR(80) NOT NULL,
  FOREIGN KEY (codigo) REFERENCES personas(codigo)
  ON DELETE CASCADE
);
  
```

// TAREAS

- 1 Crear la tabla `docentes` en MySQL con el script de arriba.
- 2 Crear **DocenteVO.java** que extienda `PersonaVO`: declarar `private String especialidad` y `private String cargo`, constructor completo con `super(codigo, nombre)`, getters y setters.

- 3 Crear **DocenteDAO.java** con los 5 métodos. Los que necesitan transacción son `agregarDocente()` (2 INSERTs) y `actualizarDocente()` (2 UPDATEs). Los que no necesitan transacción son `listarDocentes()` y `obtenerDocentePorCodigo()` (usan INNER JOIN) y `eliminarDocente()` (solo DELETE en *personas*, el CASCADE elimina la fila en *docentes*).
- 4 En los métodos que usan transacción: llamar a `con.setAutoCommit(false)` antes de los `executeUpdate()`, luego `con.commit()` si ambos retornan filas > 0, o `con.rollback()` si alguno falla.
- 5 Crear **DocenteControladorServlet.java** con `@WebServlet("/docente")` y las mismas acciones del Servlet de Alumno: `listar`, `agregar` (GET+POST), `cargarEditar` (GET), `editar` (POST), `eliminar` (POST).
- 6 Crear **docente-form.jsp**: campos para código, nombre, especialidad y cargo.
- 7 Crear **docente-lista.jsp**: tabla con columnas Código, Nombre, Especialidad, Cargo, y botones de Editar/Eliminar por fila.
- 8 Crear **docente-editar.jsp**: igual que el formulario de alta, pero con todos los campos precargados con `${docente.nombre}`, `${docente.especialidad}` y `${docente.cargo}`. El campo código debe ser `readonly`.

// ARCHIVOS A CREAR

● ● ● Estructura esperada

Base de datos:

```
tabla docentes (codigo FK → personas, especialidad, cargo)
```

Capa Modelo:

```
src/main/java/modelo/vo/DocenteV0.java
src/main/java/modelo/dao/DocenteDAO.java
```

Capa Controlador:

```
src/main/java/controlador/DocenteControladorServlet.java
```

Capa Vista:

```
src/main/webapp/WEB-INF/vista/docente-form.jsp
src/main/webapp/WEB-INF/vista/docente-lista.jsp
src/main/webapp/WEB-INF/vista/docente-editar.jsp
```

// TABLA DE MÉTODOS Y ESTRATEGIAS

MÉTODO	TABLAS INVOLUCRADAS	¿TRANSACCIÓN?
<code>listarDocentes()</code>	JOIN personas + docentes	No — solo lectura

MÉTODO	TABLAS INVOLUCRADAS	¿TRANSACCIÓN?
<code>obtenerDocentePorCodigo()</code>	JOIN con WHERE	No — solo lectura
<code>agregarDocente()</code>	INSERT personas → INSERT docentes	Sí — setAutoCommit + commit/rollback
<code>actualizarDocente()</code>	UPDATE personas + UPDATE docentes	Sí — setAutoCommit + commit/rollback
<code>eliminarDocente()</code>	DELETE personas (CASCADE borra docentes)	No — una sola operación

▲ REUTILIZAR EL PROYECTO DEL TEMA 4

No crear el proyecto desde cero — copiar *mini-crud-alumno-sql* y renombrar `AlumnoVO` → `DocenteVO`, `AlumnoDAO` → `DocenteDAO`, etc. El `pom.xml`, la clase `Conexion.java` y el `db.properties` se mantienen exactamente igual.

// CRITERIOS DE VERIFICACIÓN

- ☐ **Alta:** al agregar un docente, se insertan filas en *ambas* tablas. Verificar con `SELECT * FROM docentes` y `SELECT * FROM personas`.
- ☐ **Transacción:** si se fuerza un error en el segundo INSERT (por ejemplo, poniendo mal el nombre de la columna), el primer INSERT también se revierte y la tabla *personas* queda igual que antes.
- ☐ **Lista:** la tabla muestra especialidad y cargo de cada docente, obtenidos con INNER JOIN.
- ☐ **Eliminar:** al borrar un docente, la fila desaparece de *ambas* tablas (efecto del CASCADE). No queda ningún registro "huérfano" en *docentes*.
- ☐ **Herencia en Java:** `DocenteVO` hereda correctamente: `getDocente().getNombre()` funciona en el JSP sin necesidad de redefinir ese getter en `DocenteVO`.

💡 PARA IR MÁS LEJOS

En lugar de botones para navegar entre el CRUD de personas y el de docentes, investigar cómo implementar una barra de navegación (**navbar**) en el `index.jsp`.

Puntos de investigación:

- ¿Cómo se estructura un menú de navegación en HTML con la etiqueta `<nav>`?
- ¿Cómo se aplica CSS para que los ítems del menú se muestren en línea horizontal?
- ¿Cómo se reutiliza ese menú en *todos* los JSPs de la aplicación sin copiar el mismo código en cada archivo? (Pista: investigar `<jsp:include>`)